Group 3

Arithmetic Expression Evaluator in C++

User's Manual

Version 1.5

Revision History

Date	Version	Description	Author
04/01/2026	1.0	Initial creation and conversion of Deliverable 6 into GitHub using Markdown format	Ivan Kullaya
05/05/2026	1.1	Addition of Sections 1, 2, 3	Maha Jornaz
05/05/2026	1.2	Addition of Sections 5, 6, 7, 8	Greeshma Kunduri
05/08/2026	1.3	Addition of Section 4	Maha Jornaz
05/10/2026	1.4	Revised sections/Improved formatting consistency prior to submission	Ivan Kullaya
05/10/2026	1.5	Submission version of Deliverable 06	Maha Jornaz

Table of Contents

1. Purpose 4

2. Introduction 4
   - 2.1 Usage of Operators 4

3. Getting started 4
   - 3.1 How to Install and Run 4
     - 3.1.1 Features 5
     - 3.1.2 System Requirements 5

4. Advanced features 5
   - 4.1 How to use Implicit Multiplication 6
   - 4.2 How to use the History Textfile 6
   - 4.3 How to use Math Constants 6
   - 4.4 How to use Floating Point Numbers 6
     - 4.4.1 How to Evaluate Roots using Decimals 7

5. Troubleshooting 7

6. Examples of uses 8

7. Glossary of terms 8

8. FAQ 9

Test Case

1. Purpose

The purpose of the user manual document is to help students and users understand how the arithmetic evaluator operates. The user inputs an expression of their choice, and the software returns a value or an error back to the user. It handles operator precedence (PEMDAS), arithmetic expressions, and errors. The document includes the following sections: Purpose, Introduction, Getting Started, Advanced Features, Troubleshooting, Examples, Glossary of Terms, and FAQ.

2. Introduction

The software uses order of operations (PEMDAS) and supports the following operators and functions:

1. **()** is a parenthesis operator used to group numbers that need to be calculated first.
2. ****** is an exponential operator used to take a number's power.
3. ***** is a multiplication operator used to calculate the product.
4. / is a division operator used to calculate the quotient.
5. **+** is an addition operator used to calculate the sum.
6. **-** is a subtraction operator used to calculate the difference.
7. **%** is a modulo operator used to calculate the remainder.
8. **Unary +** is a unary positive operator used to explicitly indicate a positive value.
9. **Unary -** is a unary negative operator used to explicitly indicate a negative value.

2.1 Usage of Operators

Operator	Description	Expression
()	Parenthesis	(5+1)
**	Exponent	5 ** 1
*	Multiplication	5 * 1
/	Division	5 / 1
+	Addition	5 + 1
-	Subtraction	5 – 1
%	Modulo	5 % 1
Unary +	Positive unary operator	+(5)
Unary -	Negative unary operator	-(5)

3. Getting started

3.1 How to Install and Run

Make sure a C++ compiler and the make utility are installed on your system. Download or clone the source code from the GitHub repository, open a terminal in the project directory, and use the Makefile to build the program.

1. Download or clone the source code from the GitHub repository.

2. Open a terminal in the project directory.
3. Run the make command to build the program.
4. Execute the generated executable file.
    a. Example:
        i. Make
        ii. ./evaluator
5. Input any arithmetic expression.
    a. Enter an expression: 4 + 1
6. After running the expression, the result should appear.
    a. Result: 5
7. If the arithmetic expression is invalid, an error message will appear.
    a. Enter an expression: 7 / 0
    b. Error: Division by zero

3.1.1 Features

- Handles order of operations (PEMDAS)
- Defined error messages
- Supports arithmetic expressions
- Handles unary operators

3.1.2 System Requirements

- C++17 compatible compiler
- make utility
- Terminal or command prompt

4. Advanced features

Some advanced features that are included in the calculator are:

- Implicit multiplication e.g. (3 + 7)(6 / 2) is the same as (3 + 7) * (6 / 2)
- Saves previous expressions and results to a history file
    - Expressions that resulted in an error will not be included in the history file
- Math constants e.g. pi and e
- Handles floating point numbers
    - It's required to add zero to the floating point number if it's less than 1.
    - It can evaluate roots when entered using decimal exponents.

4.1 How to use Implicit Multiplication

1. Enter an expression without any multiplication symbols.
2. View the result.
    a. Example:
        i. Enter an expression: 2(1+4)
        ii. Result: 10

4.2 How to use the History Textfile

1. Enter several expressions.
2. View each result individually.
3. Open the generated/updated 'history.txt' textfile.
4. View the previous expressions displayed.
    a. Example:
        i. Enter an expression: 3 + 1
        ii. Result: 4
        iii. Enter an expression: (8*1
        iv. Result: Error: Mismatched parentheses!
        v. Enter an expression: 2 / 1
        vi. Result: 2
    b. (Open the generated/updated history.txt)
        i. 3 + 1 = 4
        ii. 2 / 1 = 2

4.3 How to use Math Constants

1. Enter an expression using a math constant.
2. View the result.
    a. Example:
        i. Enter an expression: 1 * pi
        ii. Result: 3.14159
        iii. Enter an expression: 3e
        iv. Result: 8.15485

4.4 How to use Floating Point Numbers

1. Enter an expression using a decimal value.
2. View the result.
    a. Example:
        i. Enter an expression: 0.1 + 3.1
        ii. Result: 3.2

4.4.1 How to Evaluate Roots using Decimals

1. Enter an expression that raises a value to a fractional or decimal exponent.
2. Use:
    a. ½ for square roots
    b. 1/3 for cube roots
    c. 1/n for other (n) root
3. View the result.
    a. Example:
        i. Enter an expression: 49 ** 0.5
        ii. Result: 7
        iii. Enter an expression: 27 ** (1/3)
        iv. Result: 3
        v. Enter an expression: 16 ** 0.25
        vi. Result: 2

5. Troubleshooting

1. The program doesn't run
    a. Make sure the program was built successfully using the make command. If errors appear during the build process, fix the reported issues and run make again. Also verify that you are running the correct executable file. If you use a macOS/Linux operating system, make sure you use ./ before the executable name when running the program in a terminal.

2. Invalid expression error
    a. This usually happens when something is wrong in the input. Common issues are missing operators (like 5 (2 + 3)), mismatched parentheses, or using invalid symbols like ^ or &.

3. Division by zero
    a. You get this when a number is divided by zero, like 10 / (5 - 5). Just change the expression so the denominator isn't zero.

4. Wrong result
    a. This is usually because of order of operations. Try using parentheses to make it clearer, and remember exponentiation uses **, not ^.

5. Program crashes or stops
    a. Re-run the program and check your input formatting. If it keeps happening, recompile and try again with a simpler expression.

6. Invalid decimal formatting
    a. Decimal numbers less than 1 must include a leading zero. For example, use 0.5 instead of .5.

7. History file is empty or not updating
    a. Only valid expressions are saved to history.txt. Expressions that produce errors will not be written to the file.

8. Unexpected root calculation result
    a. Make sure fractional exponents are entered correctly. For example:
        i. 49 ** 0.5 = 7
        ii. 27 ** (1/3) = 3

6. Examples of uses

Here are some examples of how the program works.

```
---------------------------------------------
Arithmetic Expression Evaluator
Enter 'q' to Quit
---------------------------------------------
Enter an expression: (3 + 2) * 4
Result: 20
---------------------------------------------
Enter an expression: 4 * (3 + 2) % 7 - 1
Result: 5
---------------------------------------------
Enter an expression: 5 (2 + 3)
Result: 25
---------------------------------------------
Enter an expression: 30 - 8 / ( 5 - 5 )
Error: Division by zero
---------------------------------------------
Enter an expression: q
Exiting program...
```

7. Glossary of terms

- Arithmetic expression: A math problem made up of numbers and operators.
- Binary operator: An operator that works on two numbers, like 3 + 2.
- CLI (Command Line Interface): A text-based way to interact with the program.
- Exponentiation: Raising a number to a power using **.
- Floating point number: A number containing a decimal value.
- Implicit multiplication: Multiplication without using the * symbol, such as 2(3 + 4).

- Operand: A value used in a mathematical operation.
- Operator: A symbol like +, -, *, /, %, or **.
- PEMDAS: Order of operations (parentheses, exponents, multiply/divide, add/subtract).
- Parser: The part of the program that understands the structure of the expression.
- Parentheses: Symbols used to group parts of an expression.
- Tokenization: Breaking input into numbers, operators, and parentheses.
- Unary operator: An operator that works on one number, like -5.

8. FAQ

This section should answer frequently asked questions about the software.

1. What operators are supported?
    a. +, -, *, /, %, and **.

2. Can I use decimals?
    a. Yes, as long as the number includes a leading zero.

3. What if I enter something wrong?
    a. The program will show an error message instead of crashing.

4. Do I need parentheses?
    a. Not always, but they help control order of operations.

5. Why doesn't ^ work?
    a. Because exponentiation uses ** in this program.

6. Can I enter multiple expressions at once?
    a. No, only one expression at a time.

7. What happens if I divide by zero?
    a. You will receive an error message.

8. Can I use spaces in expressions?
    a. Yes, spaces are ignored and optional in most expressions.

9. Does the calculator support negative numbers?
    a. Yes, unary operators allow expressions like -5 + 2.

10. Are invalid expressions saved to history.txt?
    a. No, expressions that result in errors are not saved.

11. Can I use nested parentheses?
    a. Yes, the calculator supports multiple levels of nested parentheses.